

Zinterpreter

Internals & Function Reference

Architecture, data structures and every function explained

Chapter 1 — Architecture Overview

The Zinterpreter is a single-pass C interpreter structured around one global virtual machine (VM) struct. There are no separate compilation and execution phases; instructions are parsed and executed in one forward scan of the instruction array. The high-level execution pipeline is:

Step	Function	Description
1	<code>main()</code>	Parse CLI args, load file, init VM
2	<code>read_code_from_file()</code>	Read .Zinter source, strip whitespace
3	<code>format_code()</code>	Split source into instruction array by ':'
4	<code>build_state()</code>	Map block boundaries into <code>state_stack[]</code>
5	<code>system_setup()</code>	Execute <code>#{ }</code> directives, resolve <code>exec_</code>
6	<code>parse()</code>	Main execution loop from <code>__start</code>

Alternatively, if no file is passed on the command line, **`run_test()`** is called directly to exercise the VM's data structures.

Chapter 2 — The VM Struct

All interpreter state lives in a single global **VM** **vm**. Preprocessor macros alias each field for ergonomic access (e.g. **variable_count** expands to **vm.variable_count**). The struct is zero-initialised with **memset(&vm, 0, sizeof(VM))** at program start.

Field group	Fields	Purpose
Scalar variables	variable[64], variable_count fl_variable[64], fl_variable_count char_variable[64], char_variable_count	Linked-list-style arrays of int/float/char variable structs
Arrays	array[64], array_count fl_array[64], fl_array_count char_array[64], char_array_count	1-D arrays of int / float / char
Matrices	matrix[64], matrix_count fl_matrix[64], fl_matrix_count char_matrix[64], char_matrix_count	2-D matrices (up to 64×64) of int / float / char
Registers	r[16], cr[16]	16 integer and 16 char scratch registers
Program store	program[512], line_idx_program	Flat array of instructions; line_idx_program = last valid index
State stack	state_stack[64], return_state	Block boundary table (functions, if, for, while ...)
Execution	global_ip	Instruction pointer (current line being executed)
Debug / misc	deb, place_holder, char_place_holder pt_place_holder, pt_char_place_holder	Debug flag, scratch placeholders for set_to_ calls

Data structure detail

```
struct decl_var { char name[16]; int value; decl_var* pointer; };
struct decl_fl_var { char name[16]; float value; decl_fl_var* pointer; };
struct decl_var_char { char name[16]; char value; decl_var_char* pointer; };
typedef struct { char name[16]; int array_int[64]; int size; } decl_arr;
typedef struct { char name[16]; float array_int[64]; int size; } decl_fl_arr;
typedef struct { char name[16]; char array_int[256];int size; } decl_char_arr;
typedef struct { char name[16]; int matrix_int[64][64]; int size_first,size_sec; }
decl_matr;
typedef struct { char name[16]; float matrix_fl_int[64][64]; int size_first,size_sec; }
decl_fl_matr;
typedef struct { char name[16]; char matrix_str_int[64][64]; int size_first,size_sec;}
decl_char_matr;
typedef struct { char nome_function[16]; int posizione_ritorno; int codice; int
posizione_skip; } program_state;
```

Chapter 3 — Utility & Type-Query Functions

starts_with(str, prefix) → int

Returns 1 if str begins with prefix (wraps strncmp). Used throughout parse() as the primary dispatch mechanism.

is_var(name) → int (0/1/2/3)

Searches variable[], char_variable[], fl_variable[] for name. Returns 1=int, 2=char, 3=float, 0=not found.

is_arr(name) → int (0/1/2/3)

Same logic for arrays: searches array[], char_array[], fl_array[]. Returns 1/2/3/0.

is_matrix(name) → int (0/1/2/3)

Same logic for matrices: searches matrix[], char_matrix[], fl_matrix[]. Returns 1/2/3/0.

is_function(name_or_decl) → int (0/1)

Searches state_stack[] for an entry whose nome_function matches name. Accepts both raw names and __name(args) format.

is_what(name_result[]) → void (writes result into arg)

Calls is_var_, is_arr_, is_matrix_, is_function_ in sequence. Writes a dot-separated result like "variable.i", "array.s", "matrix.l", "function.v" or "-1.-1" into name_result.

check_if_same_type(arg1[], arg2[]) → int (tru/fal)

Type compatibility table. Allows 'n' (number literal) to match 'i', 'k' (char literal) to match 'c'/'s', and identical types to match. Returns tru or fal.

types_match(l, right) → int (tru/fal)

Thin wrapper around check_if_same_type for single characters.

resolve_index(idx_str) → int

Parses idx_str as an integer literal or looks it up in variable[]. Returns -1 on error.

resolve(type, name) → void*

Constructs a token string "&type;&name;&" and delegates to get_index(). Returns typed pointer.

Chapter 4 — `get_index()`: The Core Resolver

```
void* get_index(char data_struct_name[])
```

`get_index` is the single most important function in the interpreter. It takes a token string and returns a raw pointer to the corresponding value in the VM, or NULL on error. Every read and every write of a Zinter datum goes through this function.

Resolution order

Condition tested	Action
TOKEN matches <code>&n;&N;&</code> (numeric literal)	Store N in static int <code>temp_int</code> ; return <code>&temp_int</code>
TOKEN matches <code>&k;&C;&</code> (char literal)	Store C in static char <code>temp_char</code> ; return <code>&temp_char</code>
TOKEN contains <code>][</code> (matrix access <code>&T;[r][c]&name;&</code>)	Parse row and col, recurse to resolve variable indices, walk <code>matrix[]</code> , <code>fl_matrix[]</code> , or <code>char_matrix[]</code>
TOKEN contains <code>[</code> (array access <code>&T;[idx]&name;&</code>)	Parse idx (literal or <code>&i;&var;&</code> token), walk <code>array[]</code> , <code>fl_array[]</code> , or <code>char_array[]</code>
TOKEN is <code>&T;&name;&</code> (scalar variable)	Walk <code>variable[]</code> , <code>char_variable[]</code> , or <code>fl_variable[]</code> by name

Variable indices in arrays/matrices are resolved by **recursive calls** to `get_index` with the inner token. This means you can have arbitrary nesting like `&i[&i&rowVar&][&i&colVar&]&mat&` and both indices will be resolved before the matrix lookup.

Important implementation notes

```
// Numeric and char literals use STATIC storage.
// The returned pointer is valid until the next call to get_index.
static int temp_int;
static char temp_char;
// Always check for ][ BEFORE checking for [ to correctly
// distinguish matrices from arrays.
if (strstr(token, "][") != NULL) { /* matrix */ }
else if (strchr(token, '[') != NULL) { /* array */ }
else { /* scalar variable */ }
```

Chapter 5 — Declaration Functions

```
void declare_variable(char name[], char type)
```

Appends a new entry to `variable[]` (type 'i'), `fl_variable[]` (type 'l') or `char_variable[]` (type 'c'). Initialises value to 0/0.0/'\0'. Increments the matching `_count`. Prints a warning if the count reaches `max_number_of_var` (64).

```
void declare_array(char name[], char type, int size)
```

Appends a new entry to `array[]` (type 'i'), `fl_array[]` (type 'l') or `char_array[]` (type 's'). Copies name, stores size (capped at 64), zero-initialises all cells. Increments `_count`.

```
void declare_matrix(char name[], char type, int size_one, int size_two)
```

Appends a new entry to `matrix[]` (type 'i'), `fl_matrix[]` (type 'l') or `char_matrix[]` (type 's'). Copies name, stores `size_first` and `size_sec`. Cells are zero-initialised by virtue of the VM being `memset` to 0 at startup. Increments `_count`.

Note: the char matrix branch correctly uses `char_matrix_count` to index into `char_matrix[]`. An earlier bug used a wrong counter; this was fixed.

```
void exec_int(char *text)
```

Handles instructions beginning with `int_`. Parses the type field: if it is plain 'i' → `declare_variable`; if it contains '[' → `declare_matrix` (int); if it contains '[' → `declare_array` (int). Size fields may be literals or variable tokens resolved via `get_index`.

```
void exec_char(char *text)
```

Same as `exec_int` but handles `char_` instructions for types 'c' (scalar), 's' (array), and 's[R][C]' (matrix). The matrix path calls `declare_matrix` with type 's'.

```
int get_array_size(char arr_name[], char type)
```

Returns the declared size of a char array (type 's') or int array (type 'i') by scanning the respective array list. Returns -1 if not found.

Chapter 6 — Set Functions

```
void set_to_variable(char name[], char type, float value, char cvalue)
```

Builds a token "&type;&name;&", calls `get_index` to get a pointer, then assigns `value` (cast to `int` or `float`) or `cvalue` depending on type. Prints an error if `get_index` returns `NULL`.

```
void set_to_array(char name[], char type, int index, float value, char cvalue)
```

Walks the appropriate array list (`array[]`, `fl_array[]`, `char_array[]`) to find `name`, bounds-checks `index`, and writes `value` or `cvalue` to the cell. Prints an error if the array is not found or the index is out of bounds.

```
void set_to_matrix(char name[], char type, int row, int col, float value, char cvalue)
```

Walks the appropriate matrix list (`matrix[]`, `fl_matrix[]`, `char_matrix[]`), bounds-checks both `row` and `col` against `size_first` / `size_sec`, and writes `value` or `cvalue`. Prints an error on bounds violation or if the matrix is not found.

```
void exec_set_to(char *text)
```

Dispatcher for the high-level `set_to_` instruction family. Parses the sub-type ('variable', 'array', 'matrix') and the argument string, then calls `get_index` to resolve `value/cvalue` tokens and the `index` token(s), and finally delegates to `set_to_variable` / `set_to_array` / `set_to_matrix`.

Argument format expected by `exec_set_to`:

Instruction form	Parsed args
<code>set_to_variable_ name,type,value,cvalue:</code>	4 fields; use @ as placeholder for unused slot
<code>set_to_array_ name,type,idx,value,cvalue:</code>	5 fields; index may be a <code>&n;&N;&</code> or <code>&i;&var;&</code> token
<code>set_to_matrix_ name,type,idx1,idx2,value,cvalue:</code>	6 fields; two index tokens

Chapter 7 — `exec_equal()`: Assignment Operator

```
void exec_equal(char *text)
```

Handles any instruction that contains a single `=` (but not `==`). Splits on `'='` into **left_operand** and **right_operand**, then dispatches through a decision tree based on the presence of `'['` (matrix) and `'['` (array) in each operand. The full matrix of cases is:

Left	Right	Case label	Action
matrix cell	matrix cell	matr x matr	get_index both, copy typed value
matrix cell	char literal	matr x k	assign literal char
matrix cell	number literal	matr x n	assign literal int/float
matrix cell	array cell	matr x arr	get_index array, copy
matrix cell	variable	matr x var	resolve variable, copy
array cell	matrix cell	arr x matr	get_index matrix, copy
variable	matrix cell	var x matr	get_index matrix, copy
array cell	array cell	arr x arr	get_index both, copy
array cell	char literal	arr x k	assign literal char
array cell	number literal	arr x n	assign literal int/float
array cell	variable	arr x var	resolve variable, copy
variable	array cell	var x arr	get_index array, copy
variable	char literal	var x k	assign literal char
variable	number literal	var x n	assign literal int/float
variable	__function	var x func	call exec_funarg, assign return
variable	variable	var x var	resolve both, copy

For each case `exec_equal` uses **`is_what()`** on the operand names to confirm they have the expected kind (variable / array / matrix) and **`types_match()`** to verify type compatibility before copying. On error it prints a descriptive message and returns without modifying memory.

Chapter 8 — I/O Functions

void exec_print(char *text)

The primary output function. Strips the 'print_' prefix and tries two parsing strategies:

```
// Strategy 1: explicit token print_&TYPE&NAME&
// sscanf(text, "print_&%15[^&]&%127[^&]&", type, name)
// Strategy 2: short form print_ [idx]name or print_ name
// sscanf(text, "print_ %127s", raw)
// is_what() resolves the base name, build token, call get_index
```

After identifying the token, exec_print dispatches on type to call printf with the appropriate format: %d (int), %f (float), %c (char), %s (string literal). Matrix and array cells are reached via get_index with the reconstructed token.

void exec_println(char *text)

Strips 'println_', builds a 'print_' prefixed buffer, calls exec_print, then printf("\n").

void exec_lnprint(char *text)

Prints a newline first, then delegates to exec_print.

void exec_lnprintln(char *text)

Prints a newline, delegates to exec_print, then prints another newline.

void exec_scan(char *text)

Stub — not yet implemented.

Chapter 9 — The Parsing Pipeline

```
char* read_code_from_file(const char *filename)
```

Opens the .Zinter file, reads it character by character into a heap-allocated buffer. Newlines and tabs are stripped unconditionally. Spaces outside of double-quoted strings are stripped (the function tracks an `is_string` flag toggled by unescaped `""` characters). Returns the compacted source string; caller must free() it.

```
void format_code(const char *code)
```

Splits the compacted source into the `program[]` instruction array. The delimiter is the colon `:`. Curly braces `{` and `}` also terminate an instruction and are themselves kept as their own entry (the colon is discarded but braces are preserved). Each entry gets a `line_number` equal to its index. After the loop, `line_idx_program` is decremented by 1 to correct an off-by-one from the loop's final increment.

```
void build_state()
```

Scans `program[]` looking for instructions that contain `{`. For each `{` found at index `i` it scans forward (tracking brace depth) to find the matching `}` at index `j`. It then stores a `program_state` entry:

```
state_stack[return_state].posizione_ritorno = i; // opening line
state_stack[return_state].posizione_skip = j; // closing line
state_stack[return_state].codice = return_state;
state_stack[return_state].nome_function = <derived from instruction>;
return_state++;
```

The `nome_function` is set to: `'if_'`, `'for_'`, `'while_'`, `'else_'`, `'__start'`, `'vd_NAME'` (for function declarations), `'#'` (for the system block). This table is queried by `system_setup`, `exec_funarg` and `is_function_`.

```
void parse(int idx_line_temp, int eventual_end_line)
```

The main execution loop. Sets `global_ip = idx_line_temp` and iterates while `global_ip ≤ line_idx_program` (or `≤ eventual_end_line` if provided). On each iteration it inspects `program[global_ip].instruction` and dispatches:

Condition (checked in order)	Function called
starts with <code>//</code>	skip (comment)
contains <code>==</code>	(reserved, see older build)
contains <code>=</code>	<code>exec_equal()</code>
starts with <code>#</code>	(system block — handled separately)
starts with <code>return_</code>	<code>exec_return()</code>
starts with <code>vd_</code>	(function declaration — no-op in parse)
starts with <code>__</code>	<code>exec_funarg()</code>
starts with <code>int_</code>	<code>exec_int()</code>
starts with <code>char_</code>	<code>exec_char()</code>
starts with <code>if_</code>	<code>exec_if()</code>

starts with else_	exec_else()
starts with for_	exec_for()
starts with while_	exec_while()
starts with set_to_	exec_set_to()
starts with scan_	exec_scan()
starts with print_	exec_print()
starts with Inprint_	exec_Inprint()
starts with println_	exec_println()
starts with Inprintln_	exec_Inprintln()

After each instruction `global_ip` is incremented. There is no jump table; branching (for control flow) will modify `global_ip` directly when implemented.

Chapter 10 — System Setup & Function Calls

```
int system_setup()
```

Finds the '#' entry in state_stack to determine the start and end lines of the #{ } block. Iterates those lines and handles three directives: **import_** (opens a .Zlib file — not yet functional), **debug_** (-dt enables debug output, -df disables it), and **exec_** (finds the '__start' entry in state_stack, sets global_ip to that line, and returns it). Returns -1 on error.

```
void* exec_funarg(char *name_plus_args, int op_code, char type_return)
```

Resolves a function call. Parses name_plus_args to extract the function name (from __name(args) format or a bare name). Searches state_stack for an entry with matching nome_function to obtain st_ip (start line) and end_ip (end line). Calls parse(st_ip, end_ip) to execute the function body. Return value handling is forthcoming.

```
void* exec_return(char *text)
```

Stub for return statements. Detects multiple return values by counting '-' characters. Full implementation pending.

```
int check_return(int eventual_end_line, int all_end_line)
```

Returns tru if a return statement is inside a function body (eventual_end_line != all_end_line), fal otherwise.

Chapter 11 — Control Flow (Stubs)

The following functions are declared and their block boundaries are recorded by `build_state`, but their execution logic is not yet implemented. They are listed here for completeness.

Function	Instruction prefix	Status
<code>exec_if(char *text)</code>	<code>if_</code>	stub — no execution yet
<code>exec_else(char *text)</code>	<code>else_</code>	stub — no execution yet
<code>exec_for(char *text)</code>	<code>for_</code>	stub — no execution yet
<code>exec_while(char *text)</code>	<code>while_</code>	stub — no execution yet

All four functions receive the current `global_ip` at the point they are called. The intended implementation will use the `state_stack` `posizione_skip` field to skip the block body when the condition is false, and loop back to the opening brace for `for/while`.

Chapter 12 — run_test() & main()

void run_test()

A self-contained regression test that exercises all VM data structures without loading a file. It declares int/char/float variables, 1-D arrays, and 2-D matrices of all three types, fills them with known values, then calls get_index with various token strings and prints the results alongside expected values. Run by invoking the interpreter with no arguments.

Test coverage:

Section	What is tested
Variables	declare_variable, set_to_variable, get_index for i/c/l
Arrays	declare_array, set_to_array, get_index with literal and variable index
Matrix int	declare_matrix, set_to_matrix, get_index [r][c] and [varrow][col]
Matrix char	char matrix declare, set, get
Matrix float	float matrix declare, set, get

int main(int argc, char *argv[])

Entry point. Flow:

```
memset(&vm, 0, sizeof(VM));
if (argc >= 3) {
    // validate .Zinter extension
    // set deb from -dt / -df
    // read_code_from_file() → format_code() → build_state()
    // print debug dump if deb==tru
    // system_setup() → parse(n, -1)
    // free(code)
} else if (argc == 1) {
    // print usage help
    run_test();
}
```

Passing -1 as eventual_end_line to parse() causes it to use line_idx_program as the upper bound, i.e. execute to the end of the file.

Chapter 13 — Constants & Limits

Constant	Value	Meaning
max_name_letter	16	Max chars in any variable / array / matrix name
max_number_of_array	64	Max arrays (and also matrices) of each type
max_number_of_var	64	Max scalars of each type
max_number_of_cell	64	Max cells per array or matrix dimension
max_lenght_of_string	256	Max cells in a char array
max_number_of_reg	16	Number of int registers r[] and char registers cr[]
max_number_of_concatened_condition	64	Depth of state_stack (block nesting depth)
program[] size	512	Max instructions in a program
instruction[] size	512	Max characters per instruction

All limits are compile-time constants defined with #define. Exceeding most of them causes a warning message rather than a hard error, but behaviour becomes undefined beyond the array bounds.